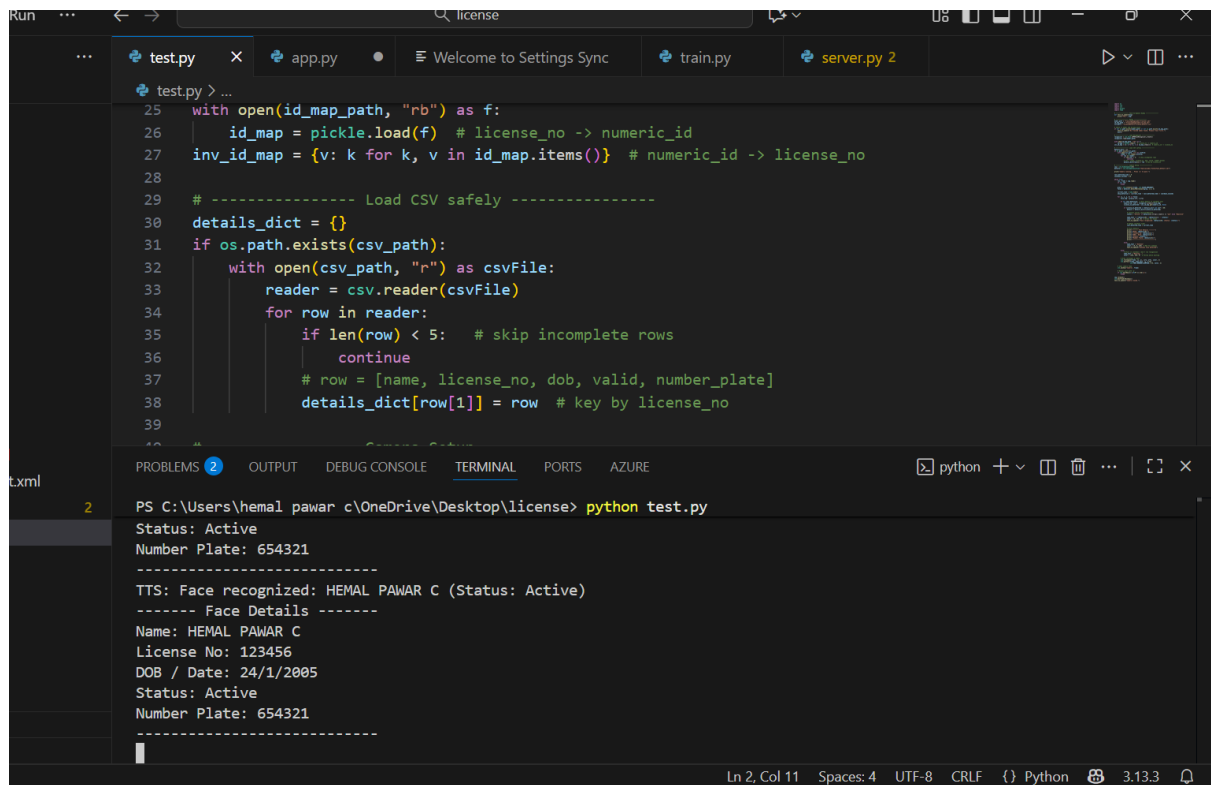


test.py ---- TESTING THE MODEL

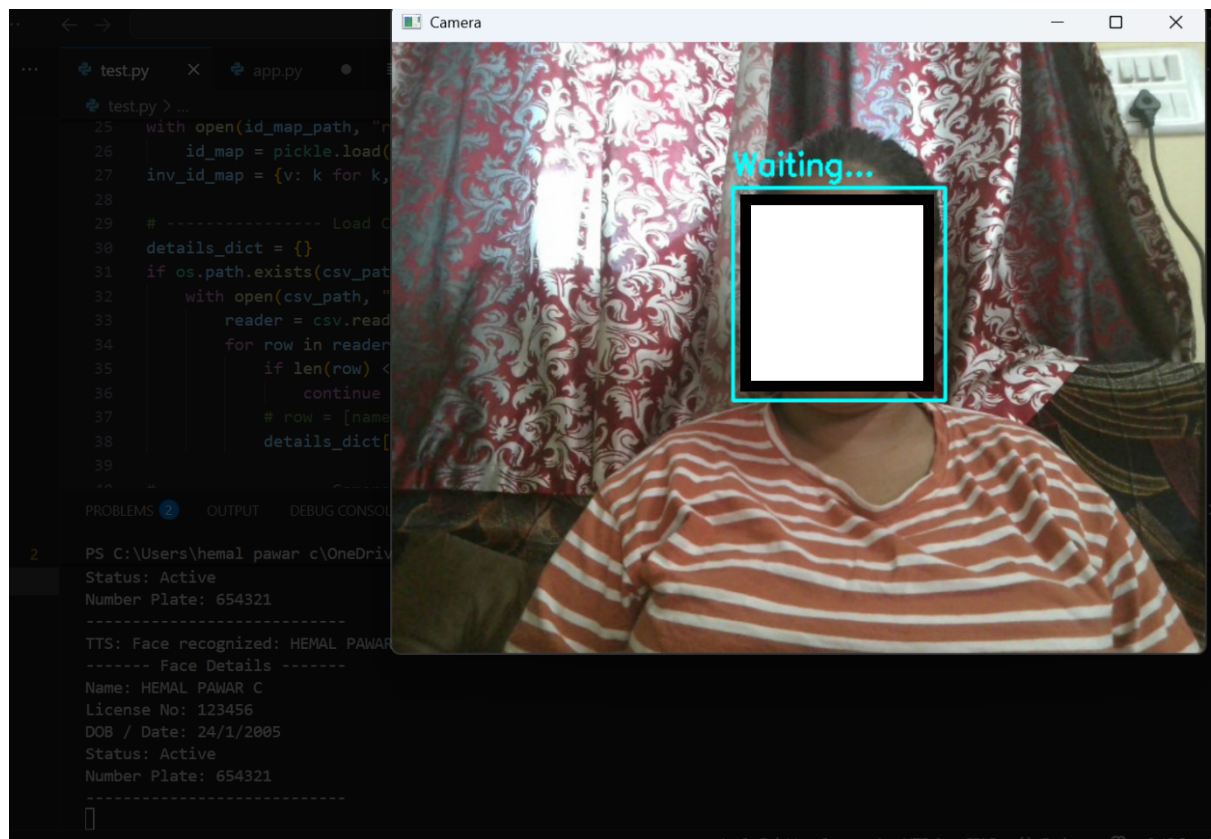


The screenshot shows a VS Code editor with the file `test.py` open. The code in `test.py` is as follows:

```
25 with open(id_map_path, "rb") as f:
26     id_map = pickle.load(f) # license_no -> numeric_id
27     inv_id_map = {v: k for k, v in id_map.items()} # numeric_id -> license_no
28
29 # ----- Load CSV safely -----
30 details_dict = {}
31 if os.path.exists(csv_path):
32     with open(csv_path, "r") as csvFile:
33         reader = csv.reader(csvFile)
34         for row in reader:
35             if len(row) < 5: # skip incomplete rows
36                 continue
37             # row = [name, license_no, dob, valid, number_plate]
38             details_dict[row[1]] = row # key by license_no
39
```

The terminal output shows the execution of `python test.py` in a PowerShell prompt:

```
PS C:\Users\hemal pawar c\OneDrive\Desktop\license> python test.py
Status: Active
Number Plate: 654321
-----
TTS: Face recognized: HEMAL PAWAR C (Status: Active)
----- Face Details -----
Name: HEMAL PAWAR C
License No: 123456
DOB / Date: 24/1/2005
Status: Active
Number Plate: 654321
-----
```



Code:

```
import os

import cv2

import csv

import pickle

import time


# ----- Text-to-Speech Dummy -----

def text_to_speech(msg):

    print("TTS:", msg)


# ----- Paths -----

model_path = "TrainingImageLabel/Trainer.yml"

id_map_path = "TrainingImageLabel/id_map.pkl"

csv_path = "LicenseDetails/license_details.csv"


# Check if model and ID map exist

if not os.path.exists(model_path) or not os.path.exists(id_map_path):

    text_to_speech("No trained model found. Please train first.")

    exit()


# ----- Load Recognizer -----

recognizer = cv2.face.LBPHFaceRecognizer_create()

recognizer.read(model_path)


with open(id_map_path, "rb") as f:

    id_map = pickle.load(f) # license_no -> numeric_id

inv_id_map = {v: k for k, v in id_map.items()} # numeric_id -> license_no
```

```

# ----- Load CSV safely -----

details_dict = {}

if os.path.exists(csv_path):
    with open(csv_path, "r") as csvFile:
        reader = csv.reader(csvFile)

        for row in reader:
            if len(row) < 5: # skip incomplete rows
                continue

            # row = [name, license_no, dob, valid, number_plate]
            details_dict[row[1]] = row # key by license_no


# ----- Camera Setup -----

cap = cv2.VideoCapture(0)
detector = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")

print("Camera running... Press 'q' to quit.")

last_detected_time = 0
cooldown_seconds = 10

while True:
    ret, frame = cap.read()

    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = detector.detectMultiScale(gray, 1.3, 5)

    current_time = time.time()
    skip_detection = (current_time - last_detected_time) < cooldown_seconds

```

```
for (x, y, w, h) in faces:
```

```
    face_img = gray[y:y+h, x:x+w]
```

```
    if not skip_detection: # Only detect if cooldown over
```

```
        numeric_id, conf = recognizer.predict(face_img)
```

```
        license_no_detected = inv_id_map.get(numeric_id, None)
```

```
    if license_no_detected in details_dict and conf < 60:
```

```
        details = details_dict[license_no_detected]
```

```
        # Convert valid → Active/Deactive
```

```
        status = "Active" if details[3].strip().lower() == "yes" else "Deactive"
```

```
        name_text = f"{details[0]} ({details[1]}) - {status}"
```

```
        color = (0, 255, 0) # Green for known
```

```
        text_to_speech(f"Face recognized: {details[0]} (Status: {status})")
```

```
        # Update cooldown timer
```

```
        last_detected_time = current_time
```

```
        # Print details
```

```
        print("----- Face Details -----")
```

```
        print(f"Name: {details[0]}")
```

```
        print(f"License No: {details[1]}")
```

```
        print(f"DOB / Date: {details[2]}")
```

```
        print(f"Status: {status}")
```

```
        print(f"Number Plate: {details[4]}")
```

```
        print("-----")
```

```
    else:
```

```
        name_text = "Unknown"
```

```
        color = (0, 0, 255) # Red for unknown
```

```
text_to_speech("Unknown face detected")

else:

    # Still in cooldown → don't run recognition
    name_text = "Waiting..."
    color = (255, 255, 0) # Yellow while waiting

    # Draw rectangle & label
    cv2.rectangle(frame, (x, y), (x+w, y+h), color, 2)
    cv2.putText(frame, name_text, (x, y-10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)

    # Show camera feed
    cv2.imshow("Camera", frame)

    # Manual quit with 'q'
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
text_to_speech("Camera closed.")
```